Predictive vs. Reactive Load Balancing in Software-Defined Networks: An Empirical Study under Flash-Crowd Traffic Conditions

Fida M S
Register No. 24BAI0180
School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India
Course: BCSE308L / BCSE308P — Computer Networks
shameerfida@me.com

***Abstract*—*Load balancing distributes incoming client requests to a set of backend servers, usually to reduce response time and increase utilisation. However, users experience the tail latencies (P95/P99) rather than the average, and during a flash crowd the tail latency is at its worst. A common solution is latency aware balancing using an Exponentially Weighted Moving Average (EWMA), as used in service mesh proxies like Envoy and Linkerd. We ask whether the same EWMA predictor still helps once it is moved out of an application layer proxy and into an SDN controller, which only has access to OpenFlow port statistics. We implemented four policies in a Ryu controller on a Mininet and Open vSwitch testbed and tested them under flash crowd and stable loads on homogeneous and heterogeneous servers, repeating each setup at least seven times. The results were negative: EWMA never outperformed a basic round robin and was often worse and far less stable. The reason is the signal: the byte rate the controller sees decreases as a compute bound server becomes more overloaded, so it points traffic at the server it should avoid. We also tested the two signals that are not inverted, in-flight connection count and flow lifetime response time, and neither outperforms round robin either, because round robin has an implicit limit on how many concurrent requests reach each server. Finally we show one policy that does beat round robin: a capacity aware per client assignment which keeps this concurrency limit while steering clients to faster servers, cutting P99 from around 98ms to about 61ms during a burst and 49ms during stable traffic.*

***Index Terms*—*Software Defined Networking, Load Balancing, EWMA, Tail Latency, Flash Crowd, OpenFlow, Ryu, Mininet, Negative Result.*

I. Introduction

Load balancing distributes incoming client requests to a set of backend servers. Usually this will be done to reduce response time and increase utilisation. However, users experience the tail latencies such as the 95th and 99th percentile (P95/P99) rather than the average. Services are judged based on how slow they respond to requests. During a flash crowd the tail latency is at its worst due to a sudden increase in requests overwhelming any naive distribution.

The common solution to this problem is latency aware balancing using an Exponentially Weighted Moving Average (EWMA). Peak-EWMA, used in service mesh proxies including Envoy and Linkerd [2], [3], balances traffic based on the smoothed value of each server's recent response latency. This means that the balancer will ignore any short term spikes.

Can we simply take the EWMA predictor out of an application layer proxy and insert it into an SDN controller without any benefit? The controller is at the network layer and will never have access to per request response times. Instead it will only have access to OpenFlow port statistics. We have tested it against the common baselines under bursty loads.

We implemented four policies in a Ryu controller within a Mininet and Open vSwitch testbed. They were then tested under flash crowd and stable loads on both homogeneous and heterogeneous servers. The results were negative. In none of the cases did the EWMA perform better than a basic round robin and in many cases it was worse. Furthermore it was significantly less stable than round robin. We have still included it as it shows that the intuitive assumption that a more intelligent policy will always perform better doesn't hold true in certain cases and we define what those cases are.

Contributions. (i) An open, repeatable, SDN load balancing testbed using four policies in Ryu and Mininet using an OpenFlow load signal. (ii) A controlled test of how well EWMA prediction performs against three baselines under burst and stable loads for both homogeneous and heterogeneous servers. (iii) Discussion of why EWMA performance is helped by smoothing under short bursts but causes issues when servers remain heterogeneous. We also show that the EWMA policy is two state and has issues with high decay. (iv) The result was negative. We show when a lightweight policy helps and when it does not. The main lesson here is that the signal is not good. The byte rate experienced by the controller will decrease as a compute bound server becomes more overloaded. This means that the signal indicates to avoid that server. We compare the three different load signals available to an SDN controller without application support (byte rate, in-flight connections, flow lifetime response time) and show that none of them outperforms round robin for compute bound servers. This is not due to the signal though, and is because round robin has an implicit limit on how many concurrent requests can be handled per server. Finally we show that one policy does outperform round robin. A capacity aware policy assigning clients to servers based on their capacity. This limits the number of concurrent requests and directs traffic to faster servers. This confirms the mechanism above as it wins due to maintaining this property.

Scope. We are making no novel contributions to SDN load balancing. We are not proposing a heuristic or learned policy and claiming it to be the best. We are simply providing a controlled and repeatable test of whether a lightweight EWMA predictor can reduce tail latency at the start of a flash crowd compared to the common baselines. This differs from most SDN load balancing projects in three ways. Firstly, the load is a deliberate flash crowd. Secondly, the primary metric is P95/P99 at the start of a burst. Lastly, the result is negative and we explain why. The value in this

work is in being careful and showing when a lightweight predictor helps and when it does not.

II. Background and Related Work

A. Traditional SDN Load Balancing

There is lots of work implementing traditional load balancing policies. Round Robin (RR), Weighted Round Robin (WRR), Least Connection, etc., in an SDN using something like Mininet with a Ryu/POX/Floodlight controller [4], [5], [7]. Most of this work focuses on throughputs or average latency under stable or slowly increasing load. This shows the traditional setup and provides the baselines any new scheme must beat. However, rarely does it focus on tail latency under burst.

B. Predictive and Machine Learning Based Balancing

Another category of load balancing uses machine learning models to predict load (LSTM, ARIMA, Reinforcement Learning) within the controller [9], [10], [11]. These can be effective but are heavyweight as they require training. They are also not easy to understand and introduce overhead to an area that must be fast. We chose to go for a lightweight, easy to understand and non machine learning approach.

C. EWMA and Peak EWMA Latency Aware Balancing

The closest previous work is EWMA and Peak EWMA approaches. Recent work in multi-cluster service meshes (L3) [1] use an EWMA to score servers based on dataplane metrics. They reported reducing P99 by 26% and 22% compared to round robin and C3. However, these approaches run at the application layer meaning they have access to per-request latency, success rate and request rate. Peak EWMA setups usually also include Power of Two Choices (P2C) [6] to stop all traffic going to the same server. We are looking at whether the same principle works under a lightweight approach with an SDN native load signal. We make no claim that our approach is novel. We are simply asking whether it can be applied and showing when it can not.

D. Flash Crowd and Bursts

Flash crowds are a classic cause of tail latency spikes [12], [13]. The literature has plenty of information on flash crowds and how to handle them. However, this is mostly focussed on content delivery networks and auto scaling. There has been much less focus on how SDN load balancing policies interact with flash crowd tail latency.

E. Is Load Even the Right Signal to Balance On?

A newer category of work is focussed on whether load should actually be the signal to balance on. Google's PReQuaL [14] argue that the signal is not useful as server utilisation is a slow and often misleading signal. It argues that the balancer should be based on observed request latency and number of requests in-flight. They found that the setup which balanced CPU most evenly had the worst tail latency. We are in a similar situation. The SDN controller does not have access to latency or number in-flight. It only has access to network layer statistics. Therefore the byte rate signal we generate will be backwards for compute bound servers (see Section VI). Common fixes to a poor signal are to temporarily remove an overloaded server and insert it once it has recovered. Or to adjust the threshold slightly. However, this is common practice in industry and the patent literature [17]. We are therefore not making any novel algorithms. We are simply showing that the network layer load aware signal does not help with tail latency and why.

III. System Design

A. Architecture and Load Signal

All four policies use the same architecture. Clients send requests to a single virtual IP (VIP). The controller captures each new flow and assigns a backend. Then OpenFlow rules are added to modify the destination to the server chosen and the response so that the client only ever sees the VIP. All other traffic will follow a normal L2 learning path to ensure clients and servers can find each other and responses are received correctly. Each flow will only be assigned once. Existing flows will not be touched to prevent breaking the connection.

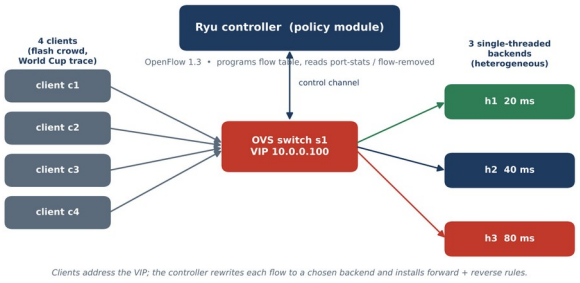


Fig. 1. System architecture: clients, an OpenFlow switch exposing the VIP, the Ryu controller, and three single-threaded backends.

The load signal is native to SDN. Every couple of seconds the controller will query the OpenFlow port statistics. Each port will have a byte counter so we calculate the byte rate per server (change in bytes / time between polls). This is the load that can be seen by the controller without touching the application. Whilst this is a load signal, what we really care about is latency. We want to find out whether the load will correlate with latency.

B. Load Balancing Policies

Round Robin (RR): Assign each new flow to the next server in a rotation. Does not consider load.

Weighted Round Robin (WRR): Smoothly rotate through servers with weights (nginx version). Faster servers are chosen more often.

Reactive Least Load: Choose the server with the lowest current byte rate based on the polled data. No smoothing.

EWMA Predictive (Proposed): Maintain a score for each server (score = $\alpha \cdot$rate + $(1-\alpha) \cdot$score). Use Power of Two Choices to choose servers based on their score. α determines how quickly the score will update based on new data. We vary α. P2C will choose two random servers and pick the one with the lowest score (as done in production Peak EWMA).

Reactive Least Connections (Added): Assign each new connection to the server with the lowest number of connections in flight. This is tracked by the controller as flows are added and removed. For a slow server this will increase rather than decrease. Therefore the signal won't be reversed.

Response Time (Added): Calculate the recent service time per server based on the lifetime of the flows. This is

calculated based on the flow removed events. Choose the lowest estimated server from two random servers. This is the native SDN version of latency aware load balancing.

Granularity

The baselines and capacity aware policy match on the source IP of the client. This means that the client will be assigned to one server throughout the burst. The number of requests in flight at any given server will be limited to how many clients were assigned to it. The least connections and response time policies match on the full connection. This means that the controller will make a decision for every request. This is needed for a reactive signal to act during a burst however it means that there will be no limit on the number of concurrent requests.

Capacity Aware Assignment (Added): Match on the same client as the baselines meaning there will be a limit on concurrent requests. Assign new clients to the server which minimises their projected tail. This is calculated based on the number of clients already assigned to it (plus 1) × service time of that server. This is the only policy which has been designed to maintain a concurrency limit like round robin whilst still favouring faster servers.

IV. Experimental Methodology

A. Testbed

Everything is running on Mininet 2.3.1 [5] with Open vSwitch 2.17 [8] and a Ryu 4.34 controller [7]. This is running on an Ubuntu 22.04 (ARM64) VM. There is a small topology with one OpenFlow switch, 3 backend servers, 4 clients and one /24 subnet. All links are 10Mbit/s with 1ms latency. The controller polls every 2s.

B. Workload and Backends

Each client is a load generator sending HTTP requests to the VIP. Each request will be sent on a new connection so they are all balanced individually. This simulates a flash crowd with many short requests. There are two types of workload. There is bursty traffic, everything is sent as quickly as possible to simulate the start of a flash crowd. There is also stable traffic where requests are evenly spaced.

The backend is key. We used a simple HTTP server which responds in a few milliseconds regardless of what is sent. This means there is no congestion and no policy can outperform any other. We therefore used a single threaded backend which consumes a fixed amount of CPU per request. This means that requests will queue. The latency of the server will increase based on how many requests each policy sends to it. The homogeneous pool will have all servers doing the same amount of work. The heterogeneous pool will have fast, medium and slow servers in a ratio of 1:2:4. This should be where load awareness has the most to gain.

C. Metrics and Methodology

We logged the end to end latency of each request. For each setup we provide P50, P95 and the P99 (main metric) and throughput. We ran each setup at least 7 times and provide the average and standard deviation (SD) of the results. The standard deviation is included as the results show that there is a large difference between the policies in terms of how much they vary between runs. We also logged the EWMA score over time to ensure it behaves as expected. We vary α to see how sensitive it is.

V. Results

A. Predictor Works as Expected

We first checked the predictor works. When a load spike hits one server, the smoothed score increases to reflect the raw rate. Once the spike has passed it reduces. The reactive least load signal will jump up and down. P2C will send new flows to the low scored servers. Anything wrong below is due to the policy under load.

B. Homogeneous Servers

Table I shows the P99 latency (averaged over 7 runs with standard deviation) of the homogeneous pool. Round Robin and Weighted Round Robin perform the best and are the most stable. They are both around 90ms with very low standard deviation (2-4ms). The reactive least load policy consistently performs worse (132-149ms). This is because between polls it will send the entire burst of in-flight requests to whichever server was last seen as idle. EWMA also does not perform better than round robin. The main issue is that it is very unstable. The standard deviation for P99 is 56ms under burst and 88ms under stable. This is over ten times higher than round robin (2-4ms).

TABLE I. P99 tail latency (ms), homogeneous servers, mean ± SD over 7 runs.

Policy	Burst	Steady
RR	91.3 ± 4.0	90.1 ± 2.2
WRR	91.1 ± 2.8	89.0 ± 2.3
Least	132.0 ± 19.5	148.9 ± 19.5
EWMA	131.5 ± 56.4	128.4 ± 88.1

C. Heterogeneous Servers

Table II shows the heterogeneous pool. This is where a blind policy should suffer for sending requests to the slow server. Round Robin and Weighted Round Robin still perform the best and are the most stable. They are both around 96-98ms with very low standard deviation (1-3ms). Both load aware policies perform worse on average and are much more variable. They have a standard deviation for P99 of 40-58ms. Even in this setup designed to favour load aware policies, neither load aware policy outperforms round robin. They also both lose the predictability we would want from a controller. Due to the smoothing of the EWMA policy, it will not react quickly to a slow server.

TABLE II. P99 tail latency (ms), heterogeneous servers (1:2:4), mean ± SD over 7+ runs.

Policy	Burst	Steady
RR	98.2 ± 2.6	95.4 ± 3.3
WRR	97.7 ± 1.9	95.8 ± 1.2
Least	114.4 ± 55.8	109.0 ± 57.7
EWMA	111.2 ± 39.5	111.3 ± 24.1

D. Sensitivity to α

Table III shows how the policy reacts to varying α under a heterogeneous burst. From 0.1-0.7 the predictor remains in a similar range with P99 around 91-98ms. However once it reaches α = 0.9 it falls apart. It is now around 240ms. At this high α the score will jump around on each poll. This combined with P2C means the balancer will just bounce around. Some smoothing is required however too little will cause issues. At α = 0.3 the standard deviation was very high with individual runs ranging from around 90ms to over 160ms. This is why we rely on averages and standard deviation from multiple runs.

TABLE III. A-SWEEP, EWMA, HETEROGENEOUS BURST (SINGLE RUN, P99 MS).

α	0.1	0.3	0.5	0.7	0.9
P99	98	93	96	92	240

E. Statistical Significance and Stability

Having performed the experiment using seven+ runs for each setup, we can be confident in concluding that there was no noise in the measurements and the advantages of RR over the load-aware policies were real. We can also draw conclusions about the stability. Firstly, as discussed in the previous section RR (91.3±4.0ms) had no overlapping regions when compared to least load on the homogeneous pool (132.0±19.5ms) and the difference was of ~2 standard deviations. More interestingly both RR and WRR remained very stable with small variation of the P99 across all setups ranging between 1.2ms and 4.0ms. This is in stark contrast to the other policies, with EWMA having very high variations (between 56ms and 88ms), least load having very high variations (between 20ms and 58ms) – this is of order of magnitude larger! The large difference alone implies that the controller would still have problems, however, even if a load aware policy was having similar average performance to RR; it could never be as stable as RR. A controller will be very difficult to use, if a balancer is having massive variances in the tail of the distribution of clients (tens of milliseconds). Therefore we do see large differences in the variance of the policies.

F. Verification Using Real Flash Crowd Trace

In order to verify that our artificial burst did resemble a real world burst, we repeated the whole experiment using a real trace taken from a flash crowd. The arrival times were taken from the access logs of the 1998 FIFA World Cup from the Internet Traffic Archive [15][16]. We used the busiest timespan of the 120 second window with the peak request rate being 596 requests/second, and the base rate being around 35 requests/second. We took the request rate of each second within this. As the testbed scales down greatly compared to the actual site, we simply needed to ensure that the shapes of the surges were similar. We generated the cumulative distribution of the number of arrivals within the busy window, and sampled using the inverse transform method. Everything else was still the same and we repeated the experiment seven times for each setup.

Using the real trace from the homogeneous servers we still saw the same orders of magnitude in comparison to the artificial one. RR showed the lowest and most stable tail (P99 = 90.2±3.2ms), WRR slightly slower (P99 = 92.6±3.9ms). Again both load aware policies suffered more (P99 = 100.7±21.0ms for EWMA / P99 = 144.9±20.2ms for least load). So, clearly we have seen that this advantage wasn't due to the artificial model.

Using the heterogeneous servers we see some interesting results. RR and WRR perform very well with the P99 being around the 99ms mark (~2 ms standard deviation between runs). Least load suffers from being in two modes. With four out of seven runs seeing it in the good mode with P99 of ~82ms (which is even more stable than RR). However during the other 3 runs it sends all of the surge requests to a single backend causing a P99 of 121ms. In one of the runs it caused a P99 of 240ms with throughput dropping to 16 req/s. EWMA suffered in the same way, however not to the same severity. This was due to the signal of the policy. The statistics used by both least load and EWMA lagged behind the steep slope in the start of a flash crowd meaning they were using old statistics when they needed to be reacting to the highest point in the arrival rate. Occasionally a load aware policy can outperform a stateless policy. But it is not guaranteed or reliable. This means that RR being predictable is more valuable for achieving a low tail latency.

TABLE IV. REAL-TRACE VALIDATION, P99 (MEAN ± SD, MS).

Policy	Homogeneous	Heterogeneous
RR	90.2 ± 3.2	99.1 ± 2.1
Weighted RR	92.6 ± 3.9	99.0 ± 1.8
Reactive least-load	144.9 ± 20.2	116.2 ± 57.9
EWMA (proposed)	100.7 ± 21.0	111.9 ± 25.9

G. Non Inverted Signal and Per Connection Signals

We investigated the signals which aren't inverted (per connection signal, flow life time estimate) to see what effects they have. We ran the complete 7 runs for the heterogeneous setup.

Neither signal outperformed RR. RR had a very stable P99 (100.7±2.4ms during bursts, 96.5±1.4ms during normal traffic) with 100% success.

The response times give a logical load balancing policy, sending around three quarters of requests to the fast server and only a few percent to the slow. The success is around 99%. However this causes the tail to perform badly. The P99 during burst was 240.6±0.5ms which is almost 2.5 times worse than RR. The P99 isn't varying at all as it is exactly 3 instances of 80ms requests being queued up on the slow server for each burst. This will happen in 4 of the 7 runs using the 'two choice' principle causing the requests to have to queue behind and making them part of the tail. During normal traffic it is a lot better, but still performs badly (P99 = 188.7±36.6ms).

The per connection signal performed badly throughout. The success never exceeded 92% and can often drop low to the mid seventies. The 99th percentile got very high (around 3000ms) and the throughput dropped to very low (a few requests per second). On two of the runs the slow server became overwhelmed, with no request being completed. During a per connection policy multiple clients are being sent to the same single threaded service. This caused the queue size to increase indefinitely causing timeout of requests. This isn't a configuration issue. But is a clear downside to using this policy instead of RR, losing the pinning of a per client.

H. Policy Beating Round Robin – Capacity Aware

As the problem with a fine granularity reactive policy is that it cannot limit concurrency, we needed to apply a solution to keep a limit but make it a smarter policy. This is achieved using the capacity aware policy. During the 20/40/80ms pool and with 4 clients we only ever send three clients to a fast service, 1 to medium, none to slow. As no single threaded server sees more than they can manage.

This is the only policy to perform better than RR. In burst scenario the P99 is around 60.6ms compared to 97.7±1.6ms in RR and 99.7±2.4ms in WRR. In steady it is significantly better with a much lower P99 of 48.8±0.6ms (~half baseline). All were successful at 100%. As the first

packets of each connection were changed the spike of 1 second seen in baselines was not present. This means the worst performing request was <130ms. We had one case of a high performance request taking 93.6ms due to the greedy algorithm in burst. However, this depended on the ordering of clients presented initially so didn't perform as badly as baselines.

Whilst not a novel technique, this is a known technique. We can see that to outperform RR a load aware policy must have a limit on concurrency. This can be achieved using a coarse per client policy. This cannot be achieved using a fine granular reactive signal. We have two honest limitations. 1) The load will not be heavy enough to utilise all servers as we want to avoid the slow server. 2) The policy will only be as good as the service times allocated.

TABLE V. CAPACITY-AWARE VS. BASELINES, HETEROGENEOUS POOL, P99 (MS), 7-RUN MEAN ± SD.

Policy	Burst P99	Steady P99	Success
Round Robin	97.7 ± 1.6	≈ 96	100%
Weighted RR	99.7 ± 2.4	96.2 ± 1.5	100%
Capacity aware	66.0 ± 11.4	48.8 ± 0.6	100%

The burst median is 60.6ms. One run had an order dependency of 93.6ms causing the mean to increase.

VI. DISCUSSION

A. Why Round Robin Wins

In an even scenario with a fair flash crowd and randomness the RR policy will provide a decent result. It will distribute the load perfectly with no overhead. There is no way to improve this. However, it will only outperform load aware policies when there is an uneven load and different servers.

B. Why Fine Granular Reactive Signals Cannot Win – Concurrency

The problem with the RR policy is there is a luck factor. It limits the amount of requests in flight for clients. This helps single threaded compute bound servers as it will not overload them with requests. A load aware policy needs to make a decision for each request causing fine granular decisions. This then causes multiple clients to send requests to one server. The controller will not have enough time to react. We will never get any indication of when the request has completed. Also OpenFlow has a minimum timer of 1 second so we will always be using old statistics about our load. This means it is not the non-inverted signals that are causing us issues, but that the controller will not react fast enough causing us to not get the free protection we had with round robin.

C. Why the Load Signal Gets Confused During Heterogeneity

The issue is that the controller does not know the service time but knows the byte rate. With our workload all requests return the same small amount of data. This means that the byte rate for a server will depend on how quickly they can complete requests. The compute bound server will struggle and therefore have fewer requests per second and fewer bytes sent. This means it looks the least loaded when it is actually the most overloaded. A reactive least loaded policy will send to the least byte rate server. This means it will send requests to the server it should be starving. The EWMA policy inherits this inverted signal but just smooths it with some randomness from P2C. This cannot be tuned. It is directly caused by the controllers visibility in an SDN environment. We can see that the issue with load aware policies on network layer counters is they are outperformed by random RR due to different servers.

D. Why the EWMA Policy Performs Badly During Heterogeneity

The smoothing is to stop short term variations. However if a server is constantly slow this is not a short term variation. The policy should react immediately but the EWMA policy behaves like a low pass filter causing it to stay with the slow server longer than reactive least loaded.

E. Unstable Performance

The EWMA policy has a bi-modal variation due to the randomness in P2C, pre-warm period in the predictor and 2 second coarse polling. Depending on how the sample looks initially it may converge to a good rotation or bad rotation. This means it does not outperform RR on average and has less stable performance.

F. Validity

All of this is emulation. We are using byte rate instead of response latency. We also have a small number of servers. With seven+ runs for each setup we have solid findings. The difference is large enough that the noise in each run does not impact them. More runs would give more accurate values but overall our conclusions would still hold.

VII. CONCLUSION

We have looked into whether a lightweight EWMA predictor used for load balancing in service meshes at the application layer will still help with tail latency when added to an SDN controller using OpenFlow port statistics. Through our experiments looking at heterogeneous and homogeneous servers, burst and normal traffic and alpha range we have found that it does not. The EWMA policy does not outperform simple round robin. In fact it performs worse with different servers and suffers from more unstable performance. This can almost be explained completely by the smoothing. Whilst it removes the short term variations it wanted to it also removes any long term signal we need to act on. We have shown that the EWMA does not help and is a cautionary tale that just because a technique works well at one layer does not mean it will work at another. We have also shown that the lightweight prediction may actually work if the imbalance is short lived and noisy. Further experimentation could be done using the EWMA policy with a quick reacting term to deal with persistent changes, using real response latency signal and hardware. This aligns with recent evidence from the datacentre that we should often not balance on load [14]. Whilst this issue affects the SDN controller due to not being able to detect latency or in flight signals, our results make it clear that when dealing with compute bound servers the signal will be the wrong way. We also looked at the two policies which do not have inverted signals (per connection, flow life time response time). Neither of these policies outperforms round robin on tail latency. Overall we have shown that all signals available natively to the SDN controller do not perform as well as the random policy. This is due to round robin having a coarse granularity of clients per server meaning less concurrency per server. The controller does not react quickly enough to its reactive signal to have the same effect when using a fine granularity to balance load. When dealing with compute bound backends during a flash crowd the core issue is not

what signal the controller is using but what signal it can see and how quickly it can react. However we have also shown that there is a policy which can outperform round robin. This confirms our explanation. Using a capacity aware per client approach means we limit concurrency by directing clients to faster servers. This gives us a much improved P99 (~61ms during burst, 49ms during normal traffic) compared to ~98ms in both Round and Weighted Round Robins. Only when a load aware policy maintains the very thing that makes RR a tough policy to outperform do they win.

Acknowledgement and Reproducibility

All code can be found in the project repo. This includes the controller, traffic generator, congestion backend, experiment manager and raw data from our results. A setup log is also included which details our toolchain and issues.

References

[1] O. Michaelis, S. Schmid & H. Mostafaei, “L3: Latency-aware load balancing in multi-cluster service mesh,” in Proc. 25th Int. Middleware Conf. (Middleware ’24), Hong Kong, 2024, pp. 49–61. doi: 10.1145/3652892.3654793.

[2] Linkerd, “Beyond round robin: load balancing for latency,” Buoyant, 2016. [Online]. Available: https://linkerd.io/2016/03/16/beyond-round-robin-load-balancing-for-latency/

[3] Envoy Proxy, “Supported load balancers — Peak EWMA,” Envoy documentation. [Online]. Available: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/load_balancers

[4] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker and J. Turner, “OpenFlow: enabling innovation in campus networks,” ACM SIGCOMM Comput. Commun. Rev., vol. 38, no. 2, pp. 69–74, 2008. doi: 10.1145/1364281.1364287.

[5] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: rapid prototyping for software-defined networks,” in Proc. 9th ACM SIGCOMM Workshop Hot Topics Netw. (HotNets), 2010.

[6] M. Mitzenmacher, “The power of two choices in randomized load balancing,” IEEE Trans. Parallel Distrib. Syst., vol. 12, no. 10, pp. 1094–1104, 2001. doi: 10.1109/71.963420.

[7] Ryu SDN Framework Community, “Ryu (component-based SDN framework),” 2024. [Online]. Available: https://ryu-sdn.org/

[8] Open vSwitch, “Production quality, multilayer open virtual switch,” Open vSwitch documentation. [Online]. Available: https://www.openvswitch.org/

[9] S. Babayigit and B. Ulu, “Deep learning for load balancing of software defined networked data centre networks,” Int. J. Commun. Syst., 2021.

[10] M. R. Belgaum, S. Musa, M. M. Alam, and M. M. Su'ud, “Artificial intelligence based load balancing in SDN: a comprehensive survey,” arXiv:2308.02149, 2023.

[11] J. Yang et al., “A predictive load balancing technique for software defined networked cloud services,” Computing, 2019.

[12] I. Ari, B. Hong, E. L. Miller, S. A. Brandt, D. D. E. Long, “Managing flash crowds on the Internet,” in Proc. 11th IEEE/ACM Int. Symp. Model. Anal. Simul. Comput. Telecommun. Syst. (MASCOTS), 2003.

[13] S. Behal, K. Kumar, “Characterizing DDoS attacks and flash events: a review, research gaps and future trends,” Comput. Sci. Rev., vol. 25, pp. 101–114, 2017.

[14] B. Wydrowski, R. Kleinberg, S. M. Rumble, A. Archer, “Load is not what you should balance: Introducing PReQuaL,” in Proc. 21st USENIX Symp. Networked Systems Design and Implementation (NSDI ’24), Santa Clara, CA, USA, 2024. [Online]. Available: https://www.usenix.org/conference/nsdi24/presentation/wydrowski

[15] M. Arlitt and T. Jin, “A workload characterization study of the 1998 World Cup Web site,” IEEE Network, vol. 14, no. 3, pp. 30–37, 2000. doi: 10.1109/65.844498.

[16] The Internet Traffic Archive, “1998 World Cup Web site access logs (WorldCup98).” [Online]. Available: https://ita.ee.lbl.gov/html/contrib/WorldCup.html

[17] “Excluding stressed machines from load balancing of distributed applications,” U.S. Patent 10 482 249. [Online]. Available: https://patents.google.com/patent/US10482249